CS7 - S03: CPU-GPU Workload Partitioning

BCSE412L Parallel Computing, Fall Semester 2026-27 | Individual Technical Report

Author: Aakaash SS **Registration No.:** 23BCE0500 **Date:** 19 October 2026

Abstract

A discrete GPU leaves the multicore CPU idle, but splitting a job across both devices adds copy and synchronisation costs. This study measures how the share of work given to the CPU changes the makespan of an independent-task workload and where the optimum lies. The workload is 256, 1024 and 4096 tiles of 256x256 floats, each filtered by 1 and 16 passes of a 3x3 blur. A share of tiles from 0 to 100 percent runs on 1, 2 and 4 OpenMP threads and the rest on a Tesla T4 through CUDA. Medians of 7 repetitions are compared with CPU-serial, CPU-only and GPU-only baselines. At the largest size with 4 threads the best split sent 70% and 20% of the tiles to the CPU (passes 1 and 16 respectively), a speedup of 2.54x and 1.25x over GPU-only. Over all 18 configurations the hybrid beat the faster single device by more than noise in 11. The optimal share fell by about 40 points as passes rose from 1 to 16, and the rule f* = T_gpu / (T_cpu + T_gpu) landed in the near-optimal range in 16 of 18 configurations.

1. Introduction and Literature Survey

A discrete GPU can run a data-parallel kernel far faster than a multicore CPU, but the data must cross the PCIe bus and the CPU sits idle while it waits. The CUDA Best Practices Guide therefore advises minimising host-device transfers and overlapping the remaining ones with computation [1]. Gregg and Hazelwood show that CPU-versus-GPU comparisons depend strongly on where the data resides and whether transfer time is counted [4], and Mittal and Vetter survey workload partitioning as one of the main techniques for using both processors [2]. Qilin automates the CPU/GPU mapping with an empirically trained performance model [3]. Nozal and Bosque extend oneAPI so that a single kernel co-executes on a CPU and an integrated GPU, and report that dynamic load balancing improves efficiency over static splits [5]. Li et al. split each neural-network layer by output channels between CPU and GPU and use latency predictors to choose the split that balances the two devices [6]. Recent work also models the cost, including energy, of CPU-GPU data transfer [7].

Gap addressed. These works study runtimes, predictors or one application. This study instead isolates one question on a discrete PCIe GPU: for independent tiles, how does the best static CPU share move with arithmetic intensity, CPU thread count and problem size, and is a rule built from two single-device timings enough to find it? It extends the CPU-versus-GPU crossover measured in S02 of this batch by examining the region between the two extremes.

2. Problem, Research Question and Hypotheses

Problem. A workload of N independent image tiles must be processed with the smallest makespan, using the CPU, the GPU, or both. **Research question (CS7 S03).** Is CPU+GPU execution always better than GPU-only execution, and what split minimises the makespan? **H1:** the optimal CPU share is neither 0 nor 100 percent and falls as arithmetic intensity (passes per tile) rises, because kernel work grows with passes while copy cost per tile does not. **H2:** the throughput-proportional share f* = T_gpu / (T_cpu + T_gpu), computed from the two single-device runs, predicts the measured optimum. **H3:** hybrid execution does not always beat GPU-only; where the CPU is slow relative to the GPU the gain disappears into noise.

3. Methodology

Workload. 256, 1024 and 4096 tiles of 256x256 single-precision pixels (64, 256 and 1024 MiB of input), each processed by 1 and 16 passes of a 3x3 box blur with clamped edges inside the tile. Passes set the arithmetic intensity while the data volume stays fixed. Tiles are independent, so any subset can run on either device. **Partition.** A share f of the tiles goes to the CPU (OpenMP, 1, 2 and 4 threads, dynamic scheduling) and 1-f to the GPU (CUDA, 64-tile chunks: host-to-device copy, one kernel per pass, device-to-host copy, pinned host memory, 1 stream). The host enqueues the GPU work asynchronously, computes its own share, then waits for the GPU. f = 0 is the GPU-only baseline and f = 1 the CPU-only baseline; CPU-serial (1 thread, f = 1) is the reference for speedup. CPU shares from 0 to 100 percent were swept in 10-point steps. **Measurements.** Per run: makespan, enqueue (scheduling) time, CPU compute time and summed per-thread busy time, GPU copy and kernel times from CUDA events, and the host wait for the GPU (synchronisation). Each point was repeated 7 times, alternating sweep direction; the median is reported and the spread is shown as min-max. GPU output was checked against the CPU result for every share (maximum relative error 0.0e+00). Metrics: speedup = baseline median / makespan (baseline always named); partition efficiency = ideal makespan 1/(1/T_cpu + 1/T_gpu) divided by measured makespan; CPU utilisation = busy thread time / (threads x makespan); GPU kernel utilisation = kernel time / makespan. **Environment.** GPU: Tesla T4, memory 15360 MiB, driver 580.178.04, target sm_75, GPUs visible: 2; CPU: Intel(R) Xeon(R) CPU @ 2.00GHz, logical CPUs 4; RAM 31.3 GB total, 30.1 GB available at start; nvcc 12.8 (V12.8.93); g++ (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0. Build: nvcc -O3 with host flags -fopenmp -O3 -march=native. Source code, raw CSV and plotting scripts are in the submitted archive; its README lists the exact commands.

4. Results and Discussion

Table 1. Summary at 4 CPU threads (medians, times in ms). Best share = CPU share with the lowest median makespan; range = shares within 2% of it. Speedups name their baseline in the column header.

Passes	Tiles	CPU-only	GPU-only	Best CPU share (range)	Best	vs GPU-only	vs CPU-only	vs CPU serial	Predicted share f*
1	256	6.5	11.3	60% (60-70%)	4.5	2.49x	1.44x	3.44x	63%
1	1024	25.4	45.7	70%	18.2	2.51x	1.39x	3.58x	64%
1	4096	99.9	180.1	70%	71.0	2.54x	1.41x	3.33x	64%
16	256	78.3	20.0	20%	16.2	1.24x	4.84x	9.67x	20%
16	1024	293.0	80.3	20%	64.0	1.26x	4.58x	11.6x	22%
16	4096	1240.7	322.2	20%	258.0	1.25x	4.81x	11.1x	21%

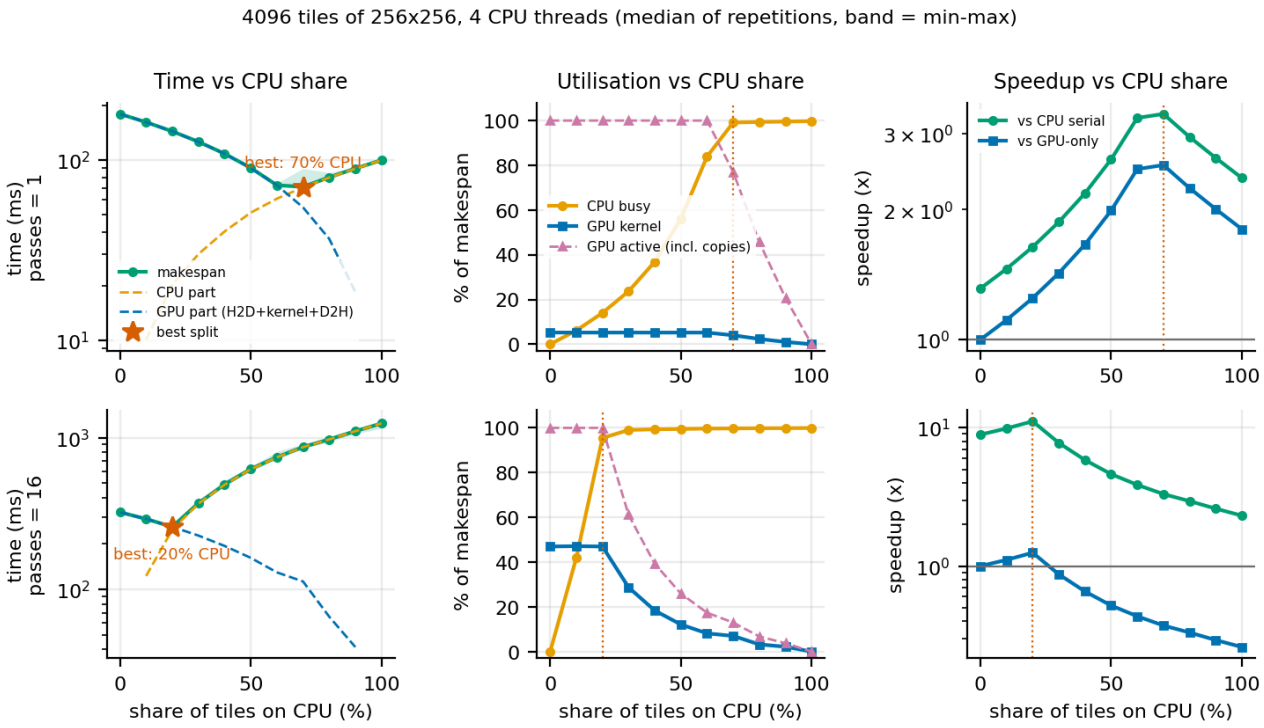


Figure 1. Makespan (with the CPU and GPU parts), utilisation and speedup against the share of tiles given to the CPU, at the largest size and thread count. Star = best share.

Headline result (4096 tiles, 4 threads). Passes 1: best share 70% CPU, 71.0 ms versus GPU-only 180 ms (2.54x) and CPU-only 99.9 ms (1.41x); Passes 16: best share 20% CPU, 258 ms versus GPU-only 322 ms (1.25x) and CPU-only 1241 ms (4.81x). Across all 18 configurations the best hybrid split beat GPU-only by more than the noise threshold (twice the larger coefficient of variation, at least 2%) in 11 and beat the faster of the two single devices in 11; the speedup of the best split over GPU-only ranged from 1.11x to 2.54x. The GPU was the faster single device in 12 of 18 configurations.

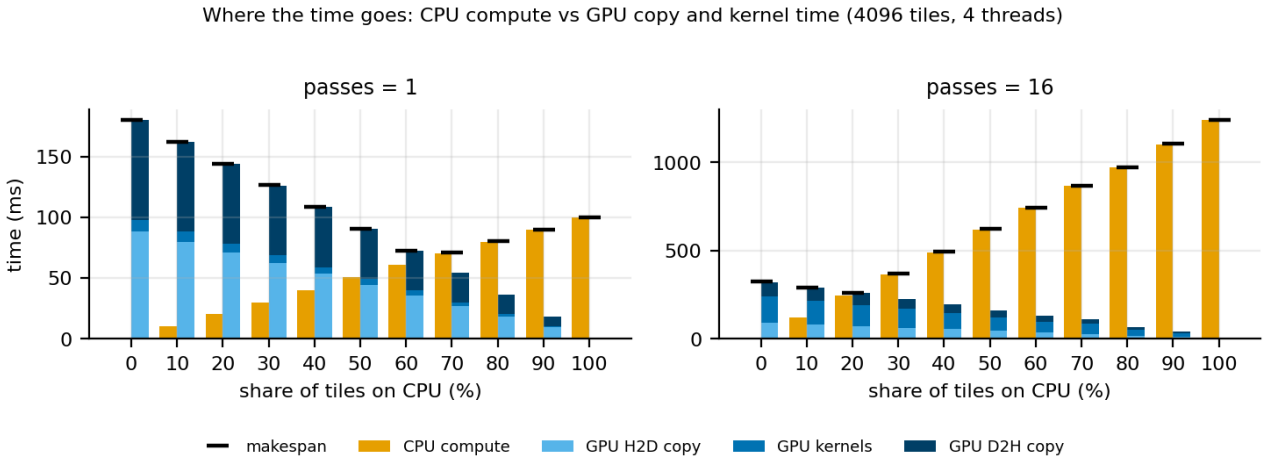


Figure 2. Time decomposition at each CPU share. Orange: CPU compute wall time. Blue stack: GPU host-to-device copy, kernels and device-to-host copy. Black dash: makespan.

Why the split behaves this way. In the GPU-only runs, copies took 95% of the GPU time at passes 1 and 53% at passes 16. Copy cost per tile is fixed while kernel cost grows with passes (Figure 2), so the GPU becomes cheaper per tile as intensity rises. Measured over all configurations, the optimal CPU share fell by about 40 points from passes 1 to 16, rose by about 19 points from 1 to 4 threads, and stayed roughly constant from 256 to 4096 tiles. At interior optima the CPU and GPU parts finished within 8% of the makespan of each other (median), and the best split reached a median 95% of the ideal perfect-overlap makespan. At the best split the host's final wait for the GPU was 1.0% and the enqueue (scheduling) time 1.1% of the makespan (medians).

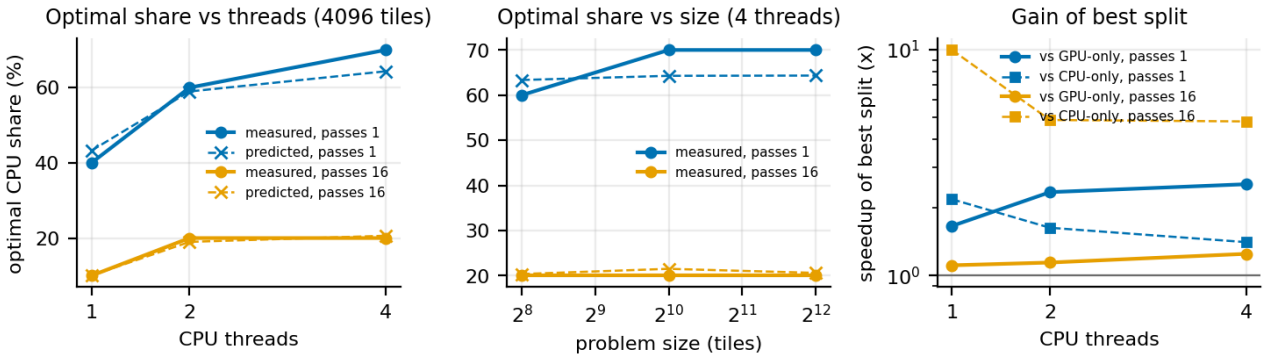


Figure 3. Optimal CPU share (solid, measured; dashed x, predicted f) against thread count and problem size, and the gain of the best split over each single device.*

Hypotheses. H1 is supported: the optimum was interior in 18 of 18 configurations and the optimal share fell by about 40 points from passes 1 to 16. H2 is supported: f* fell inside the near-optimal range (within 2% of the best makespan, plus or minus 5 points) in 16 of 18 configurations, with a mean absolute difference of 2 points from the grid optimum. H3 is supported: the hybrid did not beat GPU-only by more than noise in 7 of 18 configurations and did not beat the faster single device in 7. Run-to-run variation was a median 2.3% (maximum 16.3%) coefficient of variation of the makespan.

5. Limitations and Threats to Validity

Only one GPU model, one CPU configuration and a shared cloud notebook were used, so the optimal shares are specific to this hardware (run-to-run variation up to 16.3%). The workload is one regular, memory-light kernel; irregular kernels would shift the balance. The CPU kernel is a hand-vectorised OpenMP loop, not a tuned library, and the GPU kernel is a simple global-memory blur, so the CPU/GPU throughput ratio is implementation-specific. Device buffers are allocated once and excluded from the timing; only the host-memory type and stream count listed in Section 3 were tested. Shares were swept on a 10-point grid, so optima are resolved to about 10 points; the near-optimal range is reported for this reason. Thread counts above the physical core count share cores with the host thread that drives the GPU.

6. Conclusion and Future Work

For this tile-parallel workload CPU+GPU execution was not uniformly better: it beat the faster single device by more than noise in 11 of 18 configurations. At 4096 tiles and 4 threads the best split gave 2.54x and 1.25x over GPU-only (passes 1 and 16). The throughput-proportional rule is a usable starting point (16 of 18 configurations within the near-optimal range). Future work: (1) replace the static split with dynamic chunk scheduling or work stealing and compare it with the best static split; (2) overlap copies and kernels with several CUDA streams and measure how far the optimum moves; (3) calibrate f* online from the first few tiles, as adaptive-mapping systems do [3, 6]; (4) add energy to the objective, since transfers and idle devices change the energy-optimal split [7].

References

[1] NVIDIA, "CUDA C++ Best Practices Guide," CUDA Toolkit Documentation, v13.0. https://docs.nvidia.com/cuda/archive/13.0.0/cuda-c-best-practices-guide/index.html

[2] S. Mittal and J. S. Vetter, "A Survey of CPU-GPU Heterogeneous Computing Techniques," ACM Computing Surveys, vol. 47, no. 4, Art. 69, 2015. https://doi.org/10.1145/2788396

[3] C.-K. Luk, S. Hong, and H. Kim, "Qilin: Exploiting Parallelism on Heterogeneous Multiprocessors with Adaptive Mapping," in Proc. 42nd IEEE/ACM Int. Symp. on Microarchitecture (MICRO-42), 2009, pp. 45-55.

[4] C. Gregg and K. Hazelwood, "Where is the Data? Why You Cannot Debate CPU vs. GPU Performance Without the Answer," in Proc. IEEE ISPASS, 2011. https://doi.org/10.1109/ISPASS.2011.5762730

[5] R. Nozal and J. L. Bosque, "Exploiting co-execution with oneAPI: heterogeneity from a modern perspective," arXiv:2106.01726, 2021 (Springer version: https://doi.org/10.1007/978-3-030-85665-6_31).

[6] Z. Li, M. Paolieri, and L. Golubchik, "Accelerating Mobile Inference through Fine-Grained CPU-GPU Co-Execution," in Selected Papers of EPEW 2025, LNCS vol. 15657, Springer, 2026, pp. 41-55.

[7] F. Wang et al., "Accurate and Reliable Energy Measurement and Modelling of Data Transfer Between CPU and GPU in Parallel Applications on Heterogeneous Hybrid Platforms," IEEE Trans. Computers, vol. 74, no. 3, pp. 1011-1024, 2025. https://doi.org/10.1109/TC.2024.3504262